

FluxBoard

Architecture Notebook

1. Propósito

Este documento descreve a arquitetura do FluxBoard: objetivos, suposições, dependências, requisitos arquiteturalmente significativos, decisões, mecanismos, abstrações-chave, camadas e visões. Serve de guia para a construção e a evolução do sistema.

2. Objetivos e filosofia da arquitetura

- **Separação de responsabilidades:** organizar o sistema em camadas, isolando o domínio das tecnologias de interface e de persistência.
- **Independência de interface:** o núcleo de domínio não depende da forma de interação (textual ou gráfica), o que permite construir front-ends distintos sobre o mesmo núcleo.
- **Testabilidade:** lógica de negócio em funções/serviços puros, fáceis de testar sem UI.
- **Simplicidade:** preferir soluções simples e padrão, adequadas ao prazo e ao escopo.

3. Suposições e dependências

- Há um SGBD relacional disponível (SQLite no protótipo; PostgreSQL em produção).
- Os usuários operam em estações de trabalho padrão (Windows/Linux).
- A linguagem de implementação do protótipo é Python 3.10+.

4. Requisitos arquiteturalmente significativos

- CRUD de quadros, raias e cartões; movimentação de cartões; limite de WIP; cálculo de métricas.
- Persistência confiável e transacional.
- Registro do histórico de movimentação dos cartões (base para as métricas).
- Autenticação e autorização por projeto.
- Tempo de resposta inferior a 1 s nas operações típicas.

5. Decisões, restrições e justificativas

Decisão	Justificativa
Arquitetura em camadas (apresentação, aplicação, domínio, persistência)	Facilita manutenção, testes e troca de tecnologias.
Núcleo de domínio independente da UI	Permite TUI no protótipo e GUI/web depois sem reescrever a lógica.
Persistência relacional	Os dados são fortemente estruturados e relacionados (projetos, quadros, cartões).
Histórico de movimentação em tabela própria	As métricas exigem instantes de transição entre colunas.
SQLite no protótipo	Zero configuração; suficiente para demonstração.

6. Mecanismos de arquitetura

6.1 Mecanismo de persistência

Mapeamento das entidades de domínio para tabelas relacionais por meio de uma camada de acesso a dados (módulo `db` + serviços). Operações de escrita usam transações para garantir consistência.

6.2 Mecanismo de segurança

Autenticação por e-mail e senha, com senha armazenada como `hash` (PBKDF2/ `hashlib`). Autorização verifica a participação do usuário no projeto antes de qualquer operação.

7. Abstrações-chave

Usuario, Projeto, Participacao, Quadro, Coluna, Raia (swimlane), Cartao e MovimentacaoCartao (histórico). Estas abstrações refletem diretamente o domínio Kanban descrito nos requisitos.

8. Camadas

```
+-----+
| Apresentação (TUI no protótipo; GUI/web futura) |
+-----+
| Aplicação / Serviços (casos de uso)            |
+-----+
| Domínio (entidades e regras de negócio)        |
+-----+
| Persistência (acesso a dados / SGBD)           |
+-----+
```

A dependência é sempre de cima para baixo: a apresentação chama serviços, que usam o domínio e a persistência. O domínio não conhece a apresentação.

9. Visões da arquitetura

- **Visão lógica:** entidades de domínio e suas relações (ver projeto físico do BD).
- **Visão de implementação:** pacote `kanban` com módulos `db`, `models`, `services`, `metrics` e `tui`.
- **Visão de implantação:** ver documento de infraestrutura.

10. Impacto das ferramentas usadas

- **Python (biblioteca padrão):** acelera o desenvolvimento e reduz dependências externas; o módulo `sqlite3` fornece persistência transacional sem servidor.
- **Templates OpenUP:** padronizam os artefatos e orientam o conteúdo, reduzindo retrabalho.
- **Git + quadro Kanban:** dão rastreabilidade e visibilidade ao processo de desenvolvimento.
- **Em produção, contêineres (Docker) e PostgreSQL:** padronizam o ambiente e oferecem persistência robusta e escalável; o desacoplamento da camada de persistência permite essa troca com baixo impacto no domínio.